🤖 Machine Learning — Beginner to Expert

Complete Guide with Examples & Documentation

> Covers every ML concept from first principles through production deployment. Each section includes theory, working Python code, and real-world examples.

📋 Table of Contents

Beginner Level

1. What is Machine Learning?
2. Types of Machine Learning
3. Essential Python Libraries
4. Data Preprocessing
5. Linear Regression
6. Logistic Regression
7. Decision Trees

Intermediate Level

1. Random Forest
2. Support Vector Machines
3. K-Means Clustering
4. K-Nearest Neighbours
5. Model Evaluation Metrics
6. Feature Engineering
7. Cross-Validation

Advanced Level

1. Neural Networks from Scratch
2. Deep Learning with Keras
3. Convolutional Neural Networks (CNN)
4. Recurrent Neural Networks (RNN & LSTM)
5. Natural Language Processing (NLP)
6. Hyperparameter Tuning

Expert Level

1. Ensemble Methods — XGBoost & LightGBM
2. Transfer Learning
3. Model Deployment with FastAPI
4. MLOps & Model Monitoring
5. Interview Questions & Quick Reference

Setup — Install Everything

```
pip install numpy pandas matplotlib seaborn scikit-learn tensorflow keras \
            xgboost lightgbm fastapi uvicorn mlflow joblib nltk transformers
```

BEGINNER LEVEL

---

1. What is Machine Learning?

Machine Learning (ML) is a subset of Artificial Intelligence where computers **learn patterns from data** instead of being explicitly programmed with rules.

```
Traditional Programming:

  Rules + Data → Output




Machine Learning:

  Data + Output (examples) → Rules (model)
```

Real-world examples

Application	ML Task	Input	Output
Email spam filter	Classification	Email text	Spam / Not spam
House price predictor	Regression	Size, location	Price (£)
Netflix recommendations	Clustering	Watch history	Movie suggestions
Face unlock	Image classification	Photo	Person identity
ChatGPT	Language modelling	Text	Generated text

Your first ML program — 10 lines

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
```

```
# 1. Load data
X, y = load_iris(return_X_y=True)


# 2. Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)


# 3. Create and train the model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)


# 4. Predict and evaluate
predictions = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, predictions):.2%}")
# Accuracy: 100.00%
```

---

2. Types of Machine Learning

Supervised Learning

The model learns from **labelled examples** (input + correct output pairs).

```
# Examples: Classification and Regression

# You have: house features (input) + actual prices (labels)

# Model learns: relationship between features and price

# Predicts: price for a new house



import numpy as np

from sklearn.linear_model import LinearRegression



# Training data: [size_sqft, bedrooms] → price

X_train = np.array([[800, 2], [1200, 3], [1600, 4], [2000, 5]])

y_train = np.array([150000, 220000, 310000, 400000])



model = LinearRegression()

model.fit(X_train, y_train)



# Predict price for a 1400 sqft, 3-bedroom house

new_house = [[1400, 3]]

price = model.predict(new_house)

print(f"Predicted price: £{price[0]:,.0f}")

# Predicted price: £265,000
```

Unsupervised Learning

The model finds **hidden patterns** in unlabelled data.

```
from sklearn.cluster import KMeans

import numpy as np



# Customer spending data — no labels, just raw data

customers = np.array([

    [1000, 5],   # [annual_spend, purchase_frequency]

    [1200, 6],

    [5000, 20],

    [5500, 22],

    [200,  1],

    [300,  2],

])



# KMeans discovers 3 natural customer groups

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

kmeans.fit(customers)

print("Customer groups:", kmeans.labels_)

# Customer groups: [1 1 0 0 2 2]  — budget, premium, occasional
```

Reinforcement Learning (concept)

```
Agent (e.g., game AI) takes actions in an Environment.

Receives Rewards for good actions, Penalties for bad ones.

Learns a Policy: "in state S, take action A to maximise reward."



Example: Chess AI

  State   = current board position

  Action  = move a piece

  Reward  = +1 for winning, -1 for losing, 0 otherwise

  Goal    = learn to always choose the best move
```

---

3. Essential Python Libraries

```
import numpy as np          # fast array maths

import pandas as pd         # data manipulation (DataFrames)

import matplotlib.pyplot as plt  # plotting

import seaborn as sns       # statistical visualisation

from sklearn import *       # ML algorithms and utilities



# NumPy — array operations
```

```
arr = np.array([1, 2, 3, 4, 5])

print(arr.mean(), arr.std(), arr.max())  # 3.0  1.414  5



# Pandas — load and explore data

import pandas as pd

df = pd.read_csv('data.csv')

print(df.head())        # first 5 rows

print(df.describe())    # statistical summary

print(df.info())        # column types and null counts

print(df.isnull().sum()) # count missing values per column



# Visualisation basics

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.hist(df['price'], bins=30, color='steelblue', edgecolor='white')

plt.title('Price Distribution')



plt.subplot(1, 2, 2)

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')

plt.title('Feature Correlation')
```

```
plt.tight_layout()

plt.show()
```

---

4. Data Preprocessing

Real data is messy. Before training any model, you must clean and prepare the data.

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder

from sklearn.impute import SimpleImputer



# ── Sample messy dataset ──────────────────────────────────────────

data = {

    'age':    [25, None, 35, 40, None, 55],

    'salary': [50000, 60000, None, 80000, 90000, 70000],

    'gender': ['M', 'F', 'M', None, 'F', 'M'],

    'city':   ['London', 'Paris', 'London', 'Berlin', 'Paris', 'Berlin'],

    'bought': [1, 0, 1, 1, 0, 1]

}

df = pd.DataFrame(data)
```

```
# ── Step 1: Handle missing values ──────────────────────────────

# Numerical columns — fill with mean

num_imputer = SimpleImputer(strategy='mean')

df[['age', 'salary']] = num_imputer.fit_transform(df[['age', 'salary']])



# Categorical columns — fill with most frequent value

cat_imputer = SimpleImputer(strategy='most_frequent')

df[['gender']] = cat_imputer.fit_transform(df[['gender']])



print("After imputation:")

print(df)



# ── Step 2: Encode categorical variables ───────────────────────

# Label encoding — for ordinal categories (low < medium < high)

le = LabelEncoder()

df['gender_encoded'] = le.fit_transform(df['gender'])

# M → 1, F → 0



# One-Hot encoding — for nominal categories (no order)
```

```
df_encoded = pd.get_dummies(df, columns=['city'], drop_first=True)

# city_London, city_Paris (Berlin is the baseline — dropped)



print("\nAfter encoding:")

print(df_encoded.head())



# ── Step 3: Feature scaling ──────────────────────────────────────

# StandardScaler: (x - mean) / std → mean=0, std=1

scaler = StandardScaler()

df_encoded[['age', 'salary']] = scaler.fit_transform(

    df_encoded[['age', 'salary']])



print("\nAfter scaling:")

print(df_encoded[['age', 'salary']].describe().round(2))



# ── Step 4: Train/test split ─────────────────────────────────────

from sklearn.model_selection import train_test_split



X = df_encoded.drop('bought', axis=1)

y = df_encoded['bought']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)


print(f"\nTrain size: {len(X_train)}, Test size: {len(X_test)}")
```

---

5. Linear Regression

Predicts a **continuous numerical value** by fitting a straight line (or hyperplane) through the data.

```
Formula: y = β₀ + β₁x₁ + β₂x₂ + ... + βₙxₙ
         y = prediction,  β = coefficients,  x = features
Goal: minimise MSE (Mean Squared Error) = average of (actual - predicted)²
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score


# ── Generate house price data ──────────────────────────────
```

```
np.random.seed(42)

n = 200



house_size  = np.random.randint(500, 3000, n)

bedrooms    = np.random.randint(1, 6, n)

age_years   = np.random.randint(0, 50, n)

price       = (house_size * 150 + bedrooms * 20000

               - age_years * 1500

               + np.random.randn(n) * 20000)



df = pd.DataFrame({

    'size':     house_size,

    'bedrooms': bedrooms,

    'age':      age_years,

    'price':    price

})



# ── Train the model ──────────────────────────────────────────────

X = df[['size', 'bedrooms', 'age']]

y = df['price']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)


model = LinearRegression()
model.fit(X_train, y_train)


# ── Evaluate ──────────────────────────────────────────────────────
y_pred = model.predict(X_test)


mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)


print("=== Linear Regression Results ===")
print(f"RMSE:  £{rmse:,.0f}")
print(f"R²:    {r2:.4f}  (1.0 = perfect)")
print()
print("Coefficients:")
for feature, coef in zip(X.columns, model.coef_):
```

```
    print(f"  {feature:10s}: £{coef:+.2f} per unit")

print(f"  {'intercept':10s}: £{model.intercept_:,.2f}")



# ── Predict a new house ──────────────────────────────────────────

new_house = pd.DataFrame({'size': [1500], 'bedrooms': [3], 'age': [10]})

predicted_price = model.predict(new_house)[0]

print(f"\nPredicted price for 1500sqft, 3bed, 10yr: £{predicted_price:,.0f}")



# ── Plot actual vs predicted ─────────────────────────────────────

plt.figure(figsize=(8, 5))

plt.scatter(y_test, y_pred, alpha=0.6, color='steelblue', edgecolor='white')

plt.plot([y_test.min(), y_test.max()],

         [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect prediction')

plt.xlabel('Actual Price (£)')

plt.ylabel('Predicted Price (£)')

plt.title(f'Linear Regression: Actual vs Predicted  (R²={r2:.3f})')

plt.legend()

plt.tight_layout()

plt.show()
```

6. Logistic Regression

Despite the name, logistic regression is a **classification** algorithm. It predicts the probability that an input belongs to a class (0 or 1).

```
Output = sigmoid(z) = 1 / (1 + e^(-z))

z = β₀ + β₁x₁ + ... + βₙxₙ

Output is always between 0 and 1 — treated as probability

import numpy as np

import pandas as pd

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import (accuracy_score, classification_report,

                             confusion_matrix, roc_auc_score)

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

import seaborn as sns



# ── Loan approval dataset ──────────────────────────────────────────

np.random.seed(42)

n = 500
```

```
income      = np.random.randint(20000, 150000, n)

credit_score = np.random.randint(300, 850, n)

loan_amount = np.random.randint(5000, 50000, n)

employment_years = np.random.randint(0, 20, n)



# Approval: high income, high credit score, small loan → likely approved

approved = ((income > 60000).astype(int)

            + (credit_score > 650).astype(int)

            + (loan_amount < 25000).astype(int)

            + (employment_years > 3).astype(int))

approved = (approved >= 2).astype(int)   # approved if 2+ conditions met



df = pd.DataFrame({

    'income': income, 'credit_score': credit_score,

    'loan_amount': loan_amount, 'employment_years': employment_years,

    'approved': approved

})



# ── Prepare and train ──────────────────────────────────────────

X = df.drop('approved', axis=1)
```

```
y = df['approved']



X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y)



scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled  = scaler.transform(X_test)



model = LogisticRegression(random_state=42)

model.fit(X_train_scaled, y_train)



# ── Evaluate ──────────────────────────────────────────────────────

y_pred      = model.predict(X_test_scaled)

y_pred_prob = model.predict_proba(X_test_scaled)[:, 1]



print("=== Logistic Regression — Loan Approval ===")

print(f"Accuracy:  {accuracy_score(y_test, y_pred):.2%}")

print(f"ROC-AUC:   {roc_auc_score(y_test, y_pred_prob):.4f}")

print()
```

```
print(classification_report(y_test, y_pred,
                            target_names=['Rejected', 'Approved']))


# ── Confusion matrix ──────────────────────────────────────────────
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Rejected', 'Approved'],
            yticklabels=['Rejected', 'Approved'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.show()


# ── Feature importance (coefficients) ────────────────────────────
importance = pd.Series(model.coef_[0], index=X.columns).sort_values()
importance.plot(kind='barh', color='steelblue')
plt.title('Feature Importance (Logistic Regression Coefficients)')
plt.tight_layout()
plt.show()
```

7. Decision Trees

A tree-like model that makes decisions by asking a series of yes/no questions about the features.

```
Is income > £60,000?

├── YES → Is credit score > 650?

│         ├── YES → APPROVED ✓

│         └── NO  → Is loan < £10,000?

│                   ├── YES → APPROVED ✓

│                   └── NO  → REJECTED ✗

└── NO  → REJECTED ✗

import numpy as np

import pandas as pd

from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

from sklearn.datasets import load_iris

import matplotlib.pyplot as plt



# ── Iris flower classification ──────────────────────────────────────
```

```
iris = load_iris()

X, y = iris.data, iris.target

feature_names = iris.feature_names

class_names   = iris.target_names



X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)



# ── Train decision tree ──────────────────────────────────────────

# max_depth prevents overfitting (memorising training data)

dt = DecisionTreeClassifier(max_depth=3, random_state=42)

dt.fit(X_train, y_train)



# ── Evaluate ─────────────────────────────────────────────────────

y_pred = dt.predict(X_test)

print("=== Decision Tree — Iris Classification ===")

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")

print()

print(classification_report(y_test, y_pred, target_names=class_names))
```

```
# ── Print the tree as text ──────────────────────────────────────

print("Decision Rules:")

print(export_text(dt, feature_names=list(feature_names)))



# ── Visualise the tree ──────────────────────────────────────────

plt.figure(figsize=(14, 6))

plot_tree(dt, feature_names=feature_names, class_names=class_names,

          filled=True, rounded=True, fontsize=10)

plt.title('Decision Tree — Iris Classification')

plt.tight_layout()

plt.show()



# ── Feature importance ──────────────────────────────────────────

importance = pd.Series(dt.feature_importances_, index=feature_names)

importance.sort_values().plot(kind='barh', color='teal')

plt.title('Feature Importance')

plt.tight_layout()

plt.show()
```

INTERMEDIATE LEVEL

---

8. Random Forest

Random Forest builds **many decision trees** on random subsets of the data, then combines their predictions. More trees = less overfitting = better generalisation.

```
Why it works (ensemble principle):

  Tree 1 trained on subset A → prediction: Approved

  Tree 2 trained on subset B → prediction: Approved

  Tree 3 trained on subset C → prediction: Rejected

  ...

  100 trees vote → Majority vote → Final prediction: Approved (67 votes)

import numpy as np

import pandas as pd

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

from sklearn.datasets import load_breast_cancer

import matplotlib.pyplot as plt
```

```
# ── Breast cancer classification ──────────────────────────────

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)

y = data.target   # 0=malignant, 1=benign



X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y)



# ── Random Forest vs single Decision Tree ────────────────────────

from sklearn.tree import DecisionTreeClassifier



dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

dt_acc = accuracy_score(y_test, dt.predict(X_test))



rf = RandomForestClassifier(

    n_estimators=100,   # number of trees

    max_depth=None,     # trees can grow fully

    max_features='sqrt',# each split considers sqrt(n_features) features

    random_state=42,
```

```
    n_jobs=-1           # use all CPU cores
)
rf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test, rf.predict(X_test))


print("=== Random Forest vs Decision Tree ===")
print(f"Single Decision Tree accuracy: {dt_acc:.2%}")
print(f"Random Forest accuracy:        {rf_acc:.2%}")
print()
print(classification_report(y_test, rf.predict(X_test),
                            target_names=['Malignant', 'Benign']))


# ── Top 10 most important features ──────────────────────────────
importance = pd.Series(rf.feature_importances_,
                       index=data.feature_names).nlargest(10)


plt.figure(figsize=(9, 5))
importance.sort_values().plot(kind='barh', color='darkorange')
plt.title('Top 10 Feature Importances — Random Forest')
plt.xlabel('Importance Score')
```

```
plt.tight_layout()

plt.show()
```

---

9. Support Vector Machines

SVM finds the **optimal hyperplane** that maximally separates classes. It is especially powerful for high-dimensional data and works well when classes are linearly separable (or nearly so with the kernel trick).

```
import numpy as np

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, classification_report

from sklearn.datasets import make_classification

import matplotlib.pyplot as plt



# ── Generate 2-class dataset ──────────────────────────────────────

X, y = make_classification(n_samples=300, n_features=2, n_redundant=0,

                           n_clusters_per_class=1, random_state=42)



X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42)


# Scale features — SVM is sensitive to scale
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)


# ── Compare different SVM kernels ──────────────────────────────
kernels = ['linear', 'rbf', 'poly']
results = {}


for kernel in kernels:
    svm = SVC(kernel=kernel, C=1.0, random_state=42)
    svm.fit(X_train_s, y_train)
    acc = accuracy_score(y_test, svm.predict(X_test_s))
    results[kernel] = acc
    print(f"Kernel: {kernel:8s}  Accuracy: {acc:.2%}")


# ── Visualise decision boundary (linear kernel) ────────────────
svm_linear = SVC(kernel='linear', C=1.0)
```

```
svm_linear.fit(X_train_s, y_train)



# Create mesh grid

xx, yy = np.meshgrid(np.linspace(-3, 3, 200), np.linspace(-3, 3, 200))

Z = svm_linear.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)



plt.figure(figsize=(8, 6))

plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')

plt.scatter(X_train_s[:, 0], X_train_s[:, 1], c=y_train,

            cmap='RdBu', edgecolor='white', s=60, label='Train')

plt.scatter(X_test_s[:, 0], X_test_s[:, 1], c=y_test,

            cmap='RdBu', edgecolor='black', s=100, marker='^', label='Test')

plt.title('SVM Linear Kernel — Decision Boundary')

plt.legend()

plt.tight_layout()

plt.show()
```

---

10. K-Means Clustering

An **unsupervised** algorithm that groups data into K clusters. Each data point is assigned to the nearest cluster centre (centroid).

```
import numpy as np

import pandas as pd

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import silhouette_score

import matplotlib.pyplot as plt



# ── Customer segmentation ──────────────────────────────────────────

np.random.seed(42)

n = 300



# Simulate 3 natural customer groups

group1 = np.random.multivariate_normal([20000, 2],   [[5000**2, 0],[0, 1]], 100)

group2 = np.random.multivariate_normal([60000, 10],  [[8000**2, 0],[0, 2]], 100)

group3 = np.random.multivariate_normal([120000, 30], [[10000**2,0],[0, 3]], 100)



data = np.vstack([group1, group2, group3])

df = pd.DataFrame(data, columns=['annual_income', 'purchases_per_month'])



scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(df)



# ── Find optimal K using the elbow method ──────────────────────────

inertias    = []

silhouettes = []

K_range     = range(2, 9)



for k in K_range:

    km = KMeans(n_clusters=k, random_state=42, n_init=10)

    km.fit(X_scaled)

    inertias.append(km.inertia_)

    silhouettes.append(silhouette_score(X_scaled, km.labels_))



fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(K_range, inertias, 'bo-', lw=2)

ax1.set_xlabel('Number of Clusters (K)')

ax1.set_ylabel('Inertia (within-cluster sum of squares)')

ax1.set_title('Elbow Method — Find Optimal K')

ax1.axvline(x=3, color='red', linestyle='--', label='Optimal K=3')

ax1.legend()
```

```
ax2.plot(K_range, silhouettes, 'go-', lw=2)

ax2.set_xlabel('Number of Clusters (K)')

ax2.set_ylabel('Silhouette Score (higher = better)')

ax2.set_title('Silhouette Score')

plt.tight_layout()

plt.show()



# ── Train with optimal K=3 ──────────────────────────────────────────

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

df['cluster'] = kmeans.fit_predict(X_scaled)



# Map cluster numbers to meaningful labels

cluster_labels = {

    df.groupby('cluster')['annual_income'].mean().idxmin(): 'Budget',

    df.groupby('cluster')['annual_income'].mean().idxmax(): 'Premium',

}

remaining_cluster = [c for c in range(3) if c not in cluster_labels][0]

cluster_labels[remaining_cluster] = 'Mid-tier'

df['segment'] = df['cluster'].map(cluster_labels)
```

```
print("=== Customer Segments ===")

print(df.groupby('segment')[['annual_income',
'purchases_per_month']].mean().round(0))
```

---

11. K-Nearest Neighbours

KNN classifies a new data point by looking at its **K nearest neighbours** and taking a majority vote.

```
import numpy as np

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.datasets import load_wine

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt



data = load_wine()

X, y = data.data, data.target



X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y)


scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)

X_test_s  = scaler.transform(X_test)


# ── Find best K ──────────────────────────────────────────────

k_values   = range(1, 31)

cv_scores  = []


for k in k_values:

    knn = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(knn, X_train_s, y_train, cv=5, scoring='accuracy')

    cv_scores.append(scores.mean())


best_k = k_values[np.argmax(cv_scores)]

print(f"Best K: {best_k}  (CV accuracy: {max(cv_scores):.2%})")


plt.figure(figsize=(9, 4))

plt.plot(k_values, cv_scores, 'b-o', markersize=5)
```

```
plt.axvline(x=best_k, color='red', linestyle='--', label=f'Best K={best_k}')

plt.xlabel('K (Number of Neighbours)')

plt.ylabel('Cross-Validation Accuracy')

plt.title('KNN — Finding Optimal K')

plt.legend()

plt.tight_layout()

plt.show()



# ── Train with best K ──────────────────────────────────────────

knn_best = KNeighborsClassifier(n_neighbors=best_k)

knn_best.fit(X_train_s, y_train)

print(f"Test accuracy: {accuracy_score(y_test, knn_best.predict(X_test_s)):.2%}")
```

---

12. Model Evaluation Metrics

Choosing the right metric depends on your problem type and what type of mistake is most costly.

```
import numpy as np

from sklearn.metrics import (

    accuracy_score, precision_score, recall_score, f1_score,

    roc_auc_score, mean_squared_error, mean_absolute_error, r2_score,
```

```
    confusion_matrix, classification_report, roc_curve

)

import matplotlib.pyplot as plt



# ── Classification metrics explained ──────────────────────────────

y_true = np.array([1, 0, 1, 1, 0, 1, 0, 0, 1, 0])

y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 0, 1, 0])



print("=== Classification Metrics ===")

print(f"Accuracy:  {accuracy_score(y_true, y_pred):.2%}  (correct / total)")

print(f"Precision: {precision_score(y_true, y_pred):.2%}  (of predicted +ve, how many were right?)")

print(f"Recall:    {recall_score(y_true, y_pred):.2%}  (of actual +ve, how many did we catch?)")

print(f"F1 Score:  {f1_score(y_true, y_pred):.2%}  (harmonic mean of precision & recall)")



# ── When to use which metric ────────────────────────────────────────

print("""

When to use:

  Accuracy:  Balanced classes, no preference for false positives vs negatives
```

```
  Precision: Cost of false positives is HIGH (spam filter — don't block real
emails)

  Recall:    Cost of false negatives is HIGH (cancer detection — don't miss cases)

  F1 Score:  Imbalanced classes, need balance of precision and recall

  ROC-AUC:   Overall model discrimination ability, threshold-independent

""")



# ── Confusion matrix breakdown ──────────────────────────────────────

cm = confusion_matrix(y_true, y_pred)

print("Confusion Matrix:")

print(f"  TN={cm[0,0]}  FP={cm[0,1]}")

print(f"  FN={cm[1,0]}  TP={cm[1,1]}")

print()

print("  TN = True Negative  (correctly predicted no)")

print("  FP = False Positive (predicted yes, actually no) — Type I Error")

print("  FN = False Negative (predicted no, actually yes) — Type II Error")

print("  TP = True Positive  (correctly predicted yes)")



# ── Regression metrics ──────────────────────────────────────────────

y_true_r = np.array([250000, 320000, 180000, 420000, 290000])

y_pred_r = np.array([245000, 335000, 175000, 410000, 300000])
```

```
mae  = mean_absolute_error(y_true_r, y_pred_r)

mse  = mean_squared_error(y_true_r, y_pred_r)

rmse = np.sqrt(mse)

r2   = r2_score(y_true_r, y_pred_r)



print("\n=== Regression Metrics ===")

print(f"MAE:  £{mae:,.0f}  (average absolute error — same unit as target)")

print(f"RMSE: £{rmse:,.0f}  (penalises large errors more than MAE)")

print(f"R²:   {r2:.4f}    (1.0=perfect, 0=as good as just predicting the mean)")
```

---

13. Feature Engineering

Creating new, more informative features from existing ones — often more impactful than choosing a better algorithm.

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import PolynomialFeatures



# ── Real estate feature engineering ──────────────────────────────

df = pd.DataFrame({
```

```
    'size_sqft':   [1200, 800, 2000, 1500, 900],
    'bedrooms':    [3, 2, 4, 3, 2],
    'bathrooms':   [2, 1, 3, 2, 1],
    'age_years':   [5, 25, 2, 15, 30],
    'distance_km': [2, 8, 1, 5, 12],
    'price':       [350000, 200000, 550000, 400000, 180000]
})


# ── Create new features from existing ones ──────────────────────
# Ratio features
df['sqft_per_bedroom'] = df['size_sqft'] / df['bedrooms']
df['bath_bed_ratio']   = df['bathrooms'] / df['bedrooms']


# Interaction features
df['size_x_location'] = df['size_sqft'] * (1 / df['distance_km'])


# Binning (convert continuous → categorical)
df['age_category'] = pd.cut(df['age_years'],
                            bins=[0, 5, 15, 30, 100],
                            labels=['New', 'Modern', 'Mature', 'Old'])
```

```
# Log transformation (handles skewed distributions)
df['log_distance'] = np.log1p(df['distance_km'])


# Polynomial features (capture non-linear relationships)
poly = PolynomialFeatures(degree=2, include_bias=False)
size_poly = poly.fit_transform(df[['size_sqft']])
print("Original features:", 1)
print("Polynomial features (degree=2):", size_poly.shape[1])


print("\nEngineered features:")
print(df[['sqft_per_bedroom', 'bath_bed_ratio',
          'size_x_location', 'age_category', 'log_distance']].head())
```

---

14. Cross-Validation

Cross-validation gives a **more reliable estimate** of model performance by training and testing on multiple different splits of the data.

```
import numpy as np
from sklearn.model_selection import (cross_val_score, KFold,
                                     StratifiedKFold, GridSearchCV)
```

```python
from sklearn.ensemble import RandomForestClassifier

from sklearn.datasets import load_breast_cancer



X, y = load_breast_cancer(return_X_y=True)



# ── K-Fold Cross-Validation ────────────────────────────────────────

# Split data into K=5 folds, train on 4, test on 1, repeat 5 times

kf     = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

model  = RandomForestClassifier(n_estimators=100, random_state=42)

scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')



print("=== 5-Fold Cross-Validation ===")

for i, score in enumerate(scores, 1):

    print(f"  Fold {i}: {score:.2%}")

print(f"  Mean:   {scores.mean():.2%} ± {scores.std():.2%}")



# ── Grid Search — find best hyperparameters ────────────────────────

param_grid = {

    'n_estimators': [50, 100, 200],

    'max_depth':    [None, 10, 20],
```

```
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X, y)

print(f"\nBest parameters: {grid_search.best_params_}")
print(f"Best CV score:   {grid_search.best_score_:.2%}")
print(f"Best model:      {grid_search.best_estimator_}")
```

ADVANCED LEVEL

---

15. Neural Networks from Scratch

Building a neural network from scratch using only NumPy — to understand what Keras/PyTorch does internally.

```
import numpy as np



class NeuralNetwork:

    """

    Simple 2-layer neural network:

    Input → Hidden layer (ReLU) → Output layer (Sigmoid)

    """



    def __init__(self, input_size, hidden_size, output_size, lr=0.01):

        # Xavier initialisation — prevents vanishing/exploding gradients

        self.W1 = np.random.randn(input_size, hidden_size)  *
np.sqrt(2/input_size)

        self.b1 = np.zeros((1, hidden_size))

        self.W2 = np.random.randn(hidden_size, output_size) *
np.sqrt(2/hidden_size)

        self.b2 = np.zeros((1, output_size))

        self.lr = lr
```

```
    def relu(self, z):        return np.maximum(0, z)

    def relu_deriv(self, z):  return (z > 0).astype(float)

    def sigmoid(self, z):     return 1 / (1 + np.exp(-z))



    def forward(self, X):

        # Layer 1: linear → ReLU

        self.Z1 = X @ self.W1 + self.b1

        self.A1 = self.relu(self.Z1)

        # Layer 2: linear → Sigmoid

        self.Z2 = self.A1 @ self.W2 + self.b2

        self.A2 = self.sigmoid(self.Z2)

        return self.A2



    def compute_loss(self, y_true, y_pred):

        m   = y_true.shape[0]

        eps = 1e-8

        return -np.mean(y_true * np.log(y_pred + eps)

                        + (1 - y_true) * np.log(1 - y_pred + eps))



    def backward(self, X, y_true):
```

```
        m = X.shape[0]

        # Output layer gradients

        dZ2 = self.A2 - y_true

        dW2 = (self.A1.T @ dZ2) / m

        db2 = np.mean(dZ2, axis=0, keepdims=True)

        # Hidden layer gradients

        dA1 = dZ2 @ self.W2.T

        dZ1 = dA1 * self.relu_deriv(self.Z1)

        dW1 = (X.T @ dZ1) / m

        db1 = np.mean(dZ1, axis=0, keepdims=True)

        # Update weights (gradient descent)

        self.W2 -= self.lr * dW2;  self.b2 -= self.lr * db2

        self.W1 -= self.lr * dW1;  self.b1 -= self.lr * db1



    def train(self, X, y, epochs=1000, print_every=100):

        for epoch in range(epochs):

            y_pred = self.forward(X)

            loss   = self.compute_loss(y, y_pred)

            self.backward(X, y)

            if epoch % print_every == 0:
```

```
                acc = np.mean((y_pred > 0.5) == y)

                print(f"Epoch {epoch:5d} | Loss: {loss:.4f} | Accuracy:
{acc:.2%}")



# ── Train on XOR problem (non-linearly separable) ──────────────────

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([[0],[1],[1],[0]])   # XOR output



nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1, lr=0.1)

nn.train(X, y, epochs=5000, print_every=1000)



print("\nXOR Predictions:")

predictions = nn.forward(X)

for i, (inputs, pred, true) in enumerate(zip(X, predictions, y)):

    print(f"  {inputs} → predicted: {pred[0]:.3f}  true: {true[0]}")
```

---

16. Deep Learning with Keras

```
import numpy as np

import tensorflow as tf

from tensorflow import keras
```

```
from tensorflow.keras import layers

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.datasets import load_breast_cancer

import matplotlib.pyplot as plt



# ── Load and prepare data ──────────────────────────────────────────

X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y)



scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test  = scaler.transform(X_test)



# ── Build neural network ───────────────────────────────────────────

model = keras.Sequential([

    layers.Input(shape=(X_train.shape[1],)),



    layers.Dense(128, activation='relu'),
```

```
    layers.BatchNormalization(),    # normalise layer outputs

    layers.Dropout(0.3),            # randomly drop 30% of neurons (prevents
overfitting)



    layers.Dense(64, activation='relu'),

    layers.BatchNormalization(),

    layers.Dropout(0.2),



    layers.Dense(32, activation='relu'),



    layers.Dense(1, activation='sigmoid')   # binary output

])



model.compile(

    optimizer=keras.optimizers.Adam(learning_rate=0.001),

    loss='binary_crossentropy',

    metrics=['accuracy', 'AUC']

)



model.summary()
```

```
# ── Train ──────────────────────────────────────────────────────────
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=15, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=5, min_lr=1e-6)
]

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=200,
    batch_size=32,
    callbacks=callbacks,
    verbose=1
)

# ── Evaluate ───────────────────────────────────────────────────────
test_loss, test_acc, test_auc = model.evaluate(X_test, y_test, verbose=0)
```

```
print(f"\nTest Accuracy: {test_acc:.2%}")

print(f"Test AUC:      {test_auc:.4f}")



# ── Plot training history ──────────────────────────────────────────

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(history.history['loss'],     label='Train Loss')

ax1.plot(history.history['val_loss'], label='Val Loss')

ax1.set_title('Loss Over Epochs')

ax1.legend()



ax2.plot(history.history['accuracy'],     label='Train Accuracy')

ax2.plot(history.history['val_accuracy'], label='Val Accuracy')

ax2.set_title('Accuracy Over Epochs')

ax2.legend()

plt.tight_layout()

plt.show()
```

---

17. Convolutional Neural Networks (CNN)

CNNs are designed for **image data**. Convolutional layers detect local patterns (edges, textures, shapes) that combine into higher-level features.

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import matplotlib.pyplot as plt

import numpy as np



# ── CIFAR-10 — 60,000 images across 10 classes ────────────────────

(X_train, y_train), (X_test, y_test) = keras.datasets.cifar10.load_data()

class_names = ['airplane','automobile','bird','cat','deer',

               'dog','frog','horse','ship','truck']



# Normalise pixel values 0-255 → 0-1

X_train, X_test = X_train / 255.0, X_test / 255.0

y_train, y_test = y_train.flatten(), y_test.flatten()



print(f"Train: {X_train.shape}  Test: {X_test.shape}")

print(f"Image size: 32x32 pixels, 3 channels (RGB)")



# ── Build CNN ──────────────────────────────────────────────────────

model = keras.Sequential([
```

```
    # Block 1: learn low-level features (edges, colours)

    layers.Conv2D(32, (3,3), padding='same', activation='relu',

                  input_shape=(32,32,3)),

    layers.Conv2D(32, (3,3), activation='relu'),

    layers.MaxPooling2D(2,2),         # reduce spatial size by 2x

    layers.Dropout(0.25),



    # Block 2: learn mid-level features (shapes, textures)

    layers.Conv2D(64, (3,3), padding='same', activation='relu'),

    layers.Conv2D(64, (3,3), activation='relu'),

    layers.MaxPooling2D(2,2),

    layers.Dropout(0.25),



    # Classifier head

    layers.Flatten(),                 # convert 2D feature maps → 1D vector

    layers.Dense(512, activation='relu'),

    layers.Dropout(0.5),

    layers.Dense(10, activation='softmax')   # 10 class probabilities

])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


model.summary()


# ── Train (use subset for demo) ──────────────────────────────────
history = model.fit(
    X_train[:10000], y_train[:10000],
    validation_data=(X_test[:2000], y_test[:2000]),
    epochs=20, batch_size=64, verbose=1
)


# ── Visualise predictions ────────────────────────────────────────
predictions = model.predict(X_test[:9])
fig, axes = plt.subplots(3, 3, figsize=(8, 8))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_test[i])
```

```
    pred_class  = class_names[np.argmax(predictions[i])]

    true_class  = class_names[y_test[i]]

    colour      = 'green' if pred_class == true_class else 'red'

    ax.set_title(f"True: {true_class}\nPred: {pred_class}",

                 color=colour, fontsize=9)

    ax.axis('off')

plt.suptitle('CNN Predictions on CIFAR-10')

plt.tight_layout()

plt.show()
```

---

18. Recurrent Neural Networks (RNN & LSTM)

RNNs process **sequential data** (text, time series, audio) by maintaining a hidden state that carries information from previous steps.

```
import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import matplotlib.pyplot as plt



# ── Time series prediction — stock price ──────────────────────────
```

```
np.random.seed(42)



# Generate synthetic stock price series

t     = np.arange(0, 500)

price = 100 + 0.2*t + 20*np.sin(0.1*t) + np.random.randn(500)*5



def create_sequences(data, seq_length=30):

    """Create (input_sequence, next_value) pairs for training."""

    X, y = [], []

    for i in range(len(data) - seq_length):

        X.append(data[i:i+seq_length])

        y.append(data[i+seq_length])

    return np.array(X), np.array(y)



# Normalise

price_norm = (price - price.mean()) / price.std()



SEQ_LEN    = 30

X, y       = create_sequences(price_norm, SEQ_LEN)

X          = X.reshape(-1, SEQ_LEN, 1)   # (samples, timesteps, features)
```

```
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]

y_train, y_test = y[:split], y[split:]



# ── Build LSTM model ──────────────────────────────────────────────

model = keras.Sequential([

    layers.LSTM(64, return_sequences=True,   # return output at each step

                input_shape=(SEQ_LEN, 1)),

    layers.Dropout(0.2),

    layers.LSTM(32, return_sequences=False), # only return final step

    layers.Dropout(0.2),

    layers.Dense(16, activation='relu'),

    layers.Dense(1)                          # single value prediction

])



model.compile(optimizer='adam', loss='mse', metrics=['mae'])

model.summary()



history = model.fit(
```

```
    X_train, y_train,

    validation_data=(X_test, y_test),

    epochs=50, batch_size=32,

    callbacks=[keras.callbacks.EarlyStopping(patience=10,
restore_best_weights=True)],

    verbose=1

)



# ── Predict and visualise ──────────────────────────────────────

y_pred = model.predict(X_test).flatten()

# Denormalise

y_test_orig = y_test * price.std() + price.mean()

y_pred_orig = y_pred * price.std() + price.mean()



plt.figure(figsize=(12, 4))

plt.plot(y_test_orig[:100], label='Actual Price', color='steelblue')

plt.plot(y_pred_orig[:100], label='LSTM Prediction', color='orange',
linestyle='--')

plt.title('LSTM Stock Price Prediction')

plt.xlabel('Time Step')

plt.ylabel('Price (£)')
```

```
plt.legend()

plt.tight_layout()

plt.show()
```

---

19. Natural Language Processing (NLP)

```
import re

import numpy as np

import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences



# ── Text preprocessing pipeline ──────────────────────────────────
```

```
def clean_text(text):

    text = text.lower()

    text = re.sub(r'http\S+', '', text)         # remove URLs

    text = re.sub(r'[^a-z\s]', '', text)        # keep only letters

    text = re.sub(r'\s+', ' ', text).strip()    # normalise whitespace

    return text



# ── Sentiment analysis dataset ──────────────────────────────────

reviews = [

    "This product is absolutely amazing and works perfectly!",

    "Terrible quality, broke after one day. Waste of money.",

    "Great value for the price, very happy with purchase",

    "Disappointed. Nothing like the description. Returning it.",

    "Outstanding customer service and fast delivery!",

    "Cheap material and poor craftsmanship. Avoid.",

    "Exceeded my expectations. Will buy again!",

    "Complete rubbish. Save your money.",

    "Brilliant product, highly recommended to everyone",

    "Awful experience. Never ordering from here again."

]
```

```
labels = [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]   # 1=positive, 0=negative



# Clean text

cleaned = [clean_text(r) for r in reviews]



# ── Approach 1: TF-IDF + Logistic Regression (fast, interpretable) ─

vectorizer = TfidfVectorizer(max_features=1000, ngram_range=(1,2))

X_tfidf    = vectorizer.fit_transform(cleaned).toarray()

y          = np.array(labels)



X_train, X_test, y_train, y_test = train_test_split(

    X_tfidf, y, test_size=0.3, random_state=42)



lr_model = LogisticRegression(random_state=42)

lr_model.fit(X_train, y_train)

print("TF-IDF + LR Accuracy:", accuracy_score(y_test, lr_model.predict(X_test)))



# ── Approach 2: Deep Learning with Embeddings ──────────────────────

MAX_WORDS = 1000

MAX_LEN   = 50
```

```
EMBED_DIM = 64



tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')

tokenizer.fit_on_texts(cleaned)

sequences = tokenizer.texts_to_sequences(cleaned)

padded    = pad_sequences(sequences, maxlen=MAX_LEN, padding='post')



model = keras.Sequential([

    layers.Embedding(MAX_WORDS, EMBED_DIM, input_length=MAX_LEN),

    layers.GlobalAveragePooling1D(),

    layers.Dense(32, activation='relu'),

    layers.Dropout(0.3),

    layers.Dense(1, activation='sigmoid')

])



model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(padded, np.array(labels), epochs=30, batch_size=4,

          validation_split=0.2, verbose=0)



# ── Predict new review ──────────────────────────────────────────
```

```
new_reviews = ["This is the best thing I have ever bought!",

               "Absolute garbage, do not buy this product."]



new_cleaned  = [clean_text(r) for r in new_reviews]

new_sequences = tokenizer.texts_to_sequences(new_cleaned)

new_padded    = pad_sequences(new_sequences, maxlen=MAX_LEN, padding='post')



predictions = model.predict(new_padded, verbose=0)

for review, pred in zip(new_reviews, predictions):

    sentiment = "POSITIVE" if pred[0] > 0.5 else "NEGATIVE"

    print(f"  Review: {review[:50]}...")

    print(f"  Sentiment: {sentiment} (confidence: {pred[0]:.2%})\n")
```

---

20. Hyperparameter Tuning

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV,

                                     cross_val_score)

from sklearn.datasets import load_breast_cancer

import numpy as np
```

```
X, y = load_breast_cancer(return_X_y=True)


# ── Grid Search — exhaustive (tries every combination) ────────────

param_grid = {

    'n_estimators': [50, 100, 200],

    'max_depth':    [5, 10, None],

    'max_features': ['sqrt', 'log2']

}

# Total combinations: 3 × 3 × 2 = 18 × 5 folds = 90 fits


grid_search = GridSearchCV(

    RandomForestClassifier(random_state=42),

    param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=1

)

grid_search.fit(X, y)


print(f"Grid Search — Best params:  {grid_search.best_params_}")

print(f"Grid Search — Best score:   {grid_search.best_score_:.2%}")
```

```
# ── Random Search — faster for large parameter spaces ──────────────

param_dist = {

    'n_estimators':     np.arange(50, 300, 10),

    'max_depth':        [None, 5, 10, 15, 20, 25],

    'min_samples_split': [2, 5, 10, 20],

    'max_features':     ['sqrt', 'log2', 0.3, 0.5]

}

# Tries 20 random combinations instead of all possibilities



random_search = RandomizedSearchCV(

    RandomForestClassifier(random_state=42),

    param_dist, n_iter=20, cv=5,

    scoring='accuracy', n_jobs=-1, random_state=42, verbose=1

)

random_search.fit(X, y)



print(f"\nRandom Search — Best params: {random_search.best_params_}")

print(f"Random Search — Best score:  {random_search.best_score_:.2%}")
```

EXPERT LEVEL

---

21. Ensemble Methods — XGBoost & LightGBM

XGBoost and LightGBM are **gradient boosting** algorithms that build trees sequentially, with each tree correcting the errors of the previous one.

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.metrics import accuracy_score, roc_auc_score

from sklearn.ensemble import RandomForestClassifier

import xgboost as xgb

import lightgbm as lgb

import matplotlib.pyplot as plt



X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# ── XGBoost ──────────────────────────────────────────────────────

xgb_model = xgb.XGBClassifier(

    n_estimators=200,

    max_depth=6,

    learning_rate=0.1,        # step size — smaller = more robust, more trees
needed

    subsample=0.8,            # use 80% of data per tree

    colsample_bytree=0.8,     # use 80% of features per tree

    reg_alpha=0.1,            # L1 regularisation

    reg_lambda=1.0,           # L2 regularisation

    eval_metric='auc',

    random_state=42,

    verbosity=0

)

xgb_model.fit(X_train, y_train,

              eval_set=[(X_test, y_test)],

              verbose=False)



xgb_acc = accuracy_score(y_test, xgb_model.predict(X_test))

xgb_auc = roc_auc_score(y_test, xgb_model.predict_proba(X_test)[:, 1])
```

```
# ── LightGBM ──────────────────────────────────────────────────────

lgb_model = lgb.LGBMClassifier(

    n_estimators=200,

    num_leaves=31,            # controls tree complexity

    learning_rate=0.1,

    feature_fraction=0.8,     # colsample equivalent

    bagging_fraction=0.8,     # subsample equivalent

    bagging_freq=5,

    random_state=42,

    verbose=-1

)

lgb_model.fit(X_train, y_train,

              eval_set=[(X_test, y_test)],

              callbacks=[lgb.early_stopping(50, verbose=False)])



lgb_acc = accuracy_score(y_test, lgb_model.predict(X_test))

lgb_auc = roc_auc_score(y_test, lgb_model.predict_proba(X_test)[:, 1])



# ── Baseline comparison ──────────────────────────────────────────

rf = RandomForestClassifier(n_estimators=200, random_state=42)
```

```
rf.fit(X_train, y_train)

rf_acc = accuracy_score(y_test, rf.predict(X_test))

rf_auc = roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1])



print("=== Algorithm Comparison ===")

print(f"{'Algorithm':<15} {'Accuracy':>10} {'ROC-AUC':>10}")

print("-" * 37)

print(f"{'Random Forest':<15} {rf_acc:>10.2%} {rf_auc:>10.4f}")

print(f"{'XGBoost':<15} {xgb_acc:>10.2%} {xgb_auc:>10.4f}")

print(f"{'LightGBM':<15} {lgb_acc:>10.2%} {lgb_auc:>10.4f}")
```

---

22. Transfer Learning

Use a model pre-trained on millions of images and fine-tune it for your specific task with a small dataset.

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.applications import MobileNetV2

import numpy as np
```

```
# ── Load pre-trained MobileNetV2 (trained on ImageNet — 1M+ images) —

base_model = MobileNetV2(

    input_shape=(96, 96, 3),

    include_top=False,         # exclude the original classifier head

    weights='imagenet'         # use pre-trained weights

)



# Phase 1: Freeze base model — only train the new head

base_model.trainable = False



model = keras.Sequential([

    base_model,

    layers.GlobalAveragePooling2D(),

    layers.Dense(128, activation='relu'),

    layers.Dropout(0.3),

    layers.Dense(5, activation='softmax')   # your 5 classes

])



model.compile(optimizer='adam',

              loss='sparse_categorical_crossentropy',
```

```
              metrics=['accuracy'])



print(f"Total params:     {model.count_params():,}")

print(f"Trainable params: {sum(v.numpy().size for v in
model.trainable_variables):,}")

print(f"Frozen params:    {sum(v.numpy().size for v in
model.non_trainable_variables):,}")



# ── Simulate training on your small dataset ──────────────────────────

X_fake = np.random.rand(100, 96, 96, 3).astype(np.float32)

y_fake = np.random.randint(0, 5, 100)



# Phase 1: Train head only

history1 = model.fit(X_fake, y_fake, epochs=5, validation_split=0.2, verbose=1)



# Phase 2: Unfreeze last few layers of base for fine-tuning

base_model.trainable = True

for layer in base_model.layers[:-20]:  # keep first layers frozen

    layer.trainable = False



model.compile(
```

```
    optimizer=keras.optimizers.Adam(1e-5),   # MUCH smaller LR for fine-tuning

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)



# Phase 2: Fine-tune

history2 = model.fit(X_fake, y_fake, epochs=5, validation_split=0.2, verbose=1)
```

---

23. Model Deployment with FastAPI

```
# ── save_model.py — train and save the model ──────────────────────

import joblib

import numpy as np

from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.datasets import load_iris



iris = load_iris()

X, y = iris.data, iris.target
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)



model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_scaled, y)



joblib.dump(model,  'iris_model.pkl')

joblib.dump(scaler, 'iris_scaler.pkl')

print("Model and scaler saved!")



# ═══════════════════════════════════════════════════════════════════

# ── api.py — serve the model as a REST API ────────────────────────

from fastapi import FastAPI, HTTPException

from pydantic import BaseModel, Field

import joblib

import numpy as np



app    = FastAPI(title="Iris Classifier API", version="1.0.0")

model  = joblib.load('iris_model.pkl')

scaler = joblib.load('iris_scaler.pkl')
```

```
CLASS_NAMES = ['setosa', 'versicolor', 'virginica']


class IrisFeatures(BaseModel):
    sepal_length: float = Field(..., ge=0, le=20, example=5.1)
    sepal_width:  float = Field(..., ge=0, le=20, example=3.5)
    petal_length: float = Field(..., ge=0, le=20, example=1.4)
    petal_width:  float = Field(..., ge=0, le=20, example=0.2)


class PredictionResponse(BaseModel):
    species:     str
    class_id:    int
    confidence:  float
    probabilities: dict


@app.get("/health")
def health():
    return {"status": "healthy", "model": "RandomForest", "version": "1.0.0"}


@app.post("/predict", response_model=PredictionResponse)
```

```
def predict(features: IrisFeatures):

    try:

        X = np.array([[features.sepal_length, features.sepal_width,

                       features.petal_length, features.petal_width]])

        X_scaled    = scaler.transform(X)

        prediction  = model.predict(X_scaled)[0]

        probas      = model.predict_proba(X_scaled)[0]

        confidence  = float(probas.max())



        return PredictionResponse(

            species=CLASS_NAMES[prediction],

            class_id=int(prediction),

            confidence=round(confidence, 4),

            probabilities={

                CLASS_NAMES[i]: round(float(p), 4)

                for i, p in enumerate(probas)

            }

        )

    except Exception as e:

        raise HTTPException(status_code=500, detail=str(e))
```

```
# Run with: uvicorn api:app --reload --port 8000

# Test with: curl -X POST http://localhost:8000/predict \

#   -H "Content-Type: application/json" \

#   -d
'{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

---

24. MLOps & Model Monitoring

```
import mlflow

import mlflow.sklearn

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

from sklearn.datasets import load_breast_cancer

import numpy as np



# ── MLflow experiment tracking ──────────────────────────────────────

mlflow.set_experiment("breast_cancer_classification")



X, y = load_breast_cancer(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)


# Run experiment with different hyperparameters
configs = [
    {'n_estimators': 50,  'max_depth': 5},
    {'n_estimators': 100, 'max_depth': 10},
    {'n_estimators': 200, 'max_depth': None},
]


for config in configs:
    with mlflow.start_run():
        # Log parameters
        mlflow.log_params(config)


        # Train
        model = RandomForestClassifier(**config, random_state=42)
        model.fit(X_train, y_train)


        # Evaluate
```

```
        y_pred      = model.predict(X_test)

        y_pred_prob = model.predict_proba(X_test)[:, 1]



        metrics = {

            'accuracy':  accuracy_score(y_test, y_pred),

            'f1_score':  f1_score(y_test, y_pred),

            'roc_auc':   roc_auc_score(y_test, y_pred_prob),

            'n_features': X_train.shape[1]

        }



        # Log metrics

        mlflow.log_metrics(metrics)



        # Log model

        mlflow.sklearn.log_model(model, "model")



        print(f"Config: {config}")

        print(f"  Accuracy: {metrics['accuracy']:.2%}")

        print(f"  ROC-AUC:  {metrics['roc_auc']:.4f}")

        print()
```

```
print("View results: mlflow ui")

print("Then open:    http://localhost:5000")



# ── Data drift detection ──────────────────────────────────────────

def detect_data_drift(reference_data, new_data, threshold=0.1):

    """

    Simple drift detection using feature mean comparison.

    In production use: Evidently AI, WhyLogs, or Alibi-Detect.

    """

    ref_means   = reference_data.mean(axis=0)

    new_means   = new_data.mean(axis=0)

    drift_ratio = np.abs(ref_means - new_means) / (np.abs(ref_means) + 1e-8)



    drifted_features = np.where(drift_ratio > threshold)[0]

    drift_detected   = len(drifted_features) > 0



    return {

        'drift_detected': drift_detected,

        'drifted_features': drifted_features.tolist(),
```

```
        'max_drift': float(drift_ratio.max()),

        'mean_drift': float(drift_ratio.mean())

    }



# Simulate production data with slight drift

new_X = X_test + np.random.randn(*X_test.shape) * 0.5



result = detect_data_drift(X_train, new_X)

print(f"\nDrift Detection: {result}")
```

---

25. Interview Questions & Quick Reference

Top Interview Questions

Q1: What is the bias-variance tradeoff?

> Bias = error from wrong assumptions (underfitting). Variance = error from sensitivity to training data (overfitting). High bias → model too simple. High variance → model too complex. Goal: find the sweet spot where both are low.

Q2: What is overfitting and how do you prevent it?

> Overfitting: model memorises training data but fails on new data. Prevention: more training data, regularisation (L1/L2), dropout (neural nets), cross-validation, early stopping, simpler model.

Q3: What is regularisation?

> L1 (Lasso): adds `|weights|` to loss — drives some weights to exactly zero (feature selection). L2 (Ridge): adds `weights²` to loss — shrinks weights towards zero but rarely to exactly zero. L1+L2

> combined = ElasticNet.

Q4: What is the difference between precision and recall?

> Precision = TP / (TP + FP) — of all positives predicted, how many were actually positive? Recall = TP / (TP + FN) — of all actual positives, how many did we find? Trade-off: increasing one often decreases the other.

Q5: Explain gradient descent.

> An optimisation algorithm that minimises the loss function by iteratively moving in the direction of steepest descent (negative gradient). Learning rate controls step size. Variants: SGD (one sample), Mini-batch (subset), Adam (adaptive learning rate).

Q6: What is a confusion matrix?

> A table showing correct vs incorrect predictions for each class. Rows = actual, Columns = predicted. Diagonal = correct predictions. Off-diagonal = mistakes.

Q7: When would you use a Random Forest vs XGBoost?

> Random Forest: easy to use, robust, good default. XGBoost: better accuracy on structured tabular data, handles missing values, used in Kaggle competitions. RF trains in parallel; XGBoost trains sequentially (boosting).

Q8: What is the curse of dimensionality?

> As the number of features grows, the data becomes sparse in the high-dimensional space. Distances become meaningless. KNN and clustering struggle. Solution: dimensionality reduction (PCA, t-SNE, UMAP) or feature selection.

Q9: What is cross-validation and why use it?

> K-fold CV: split data into K folds, train on K-1, test on 1, repeat K times. Gives K accuracy estimates — average them. More reliable than a single train/test split because it uses all data for both training and testing.

Q10: What is transfer learning?

> Use a model pre-trained on a large dataset (e.g. ImageNet) as a starting point for a different but related task. Fine-tune on your specific data. Dramatically reduces data and compute requirements.

Algorithm Quick Reference

Algorithm	Type	Problem	Pros	Cons
Linear Regression	Supervised	Regression	Fast, interpretable	Only linear relationships
Logistic Regression	Supervised	Classification	Fast, probabilistic	Only linear boundaries
Decision Tree	Supervised	Both	Interpretable, no scaling	Overfits easily
Random Forest	Supervised	Both	Robust, handles overfitting	Less interpretable
SVM	Supervised	Both	Works in high dimensions	Slow on large data
KNN	Supervised	Both	Simple, no training	Slow on prediction
K-Means	Unsupervised	Clustering	Simple, fast	Need to specify K
XGBoost	Supervised	Both	State-of-the-art accuracy	Many hyperparameters
Neural Network	Supervised	Both	Very flexible	Needs lots of data
CNN	Supervised	Images	Best for images	Computationally heavy
LSTM	Supervised	Sequences	Best for sequences	Complex, slow

Metric Quick Reference

Metric	Formula	Use When
Accuracy	Correct / Total	Balanced classes
Precision	TP / (TP+FP)	False positives costly
Recall	TP / (TP+FN)	False negatives costly
F1 Score	2×(P×R)/(P+R)	Imbalanced classes

Metric	Formula	Use When
ROC-AUC	Area under ROC curve	Overall discrimination
MAE	Mean|actual-pred|	Regression, robust to outliers
RMSE	√Mean(actual-pred)²	Regression, penalise large errors
R^2	1 - SS_res/SS_tot	Regression, proportion of variance explained

---

Machine Learning is learned best by doing. Take each section, run the code, change the parameters, and observe what happens. The fastest path from beginner to expert is building real projects on real data. 🤖